CS 61C
Summer 2020

C Basics

Discussion 2: June 24, 2020

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 True or False: C is a pass-by-value language.

True. If you want to pass a reference to anything, you should use a pointer.

1.2 What is a pointer? What does it have in common to an array variable?

A pointer is just a sequence of bits, interpreted as a memory address. An array name is not a variable. An array acts like a pointer to the first element in the allocated memory for that array.

1.3 If you try to dereference a variable that is not a pointer, what will happen? What about when you free one?

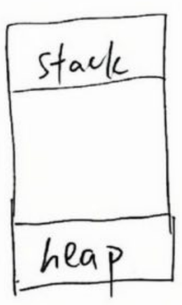
① Segmentation fault. 它会把这个变量的底层二进制位当作指针(内存地址),并尝试访问该地址处的数据.

② "invalid free" error; 释放一个已被free过的内存,或是free了一块不是通过 malloc() / calloc() / realloc() 建立的内存.

1.4 When should you use the heap over the stack? Do they grow?

① heap 是 malloc 产生的; stack 是函数局部变量.; 若需要在多次函数调用之间持续访问数据,可使用heap(传指针比传整块数据高效

② stack grows down, heap grows up, meeting when working memory is full.



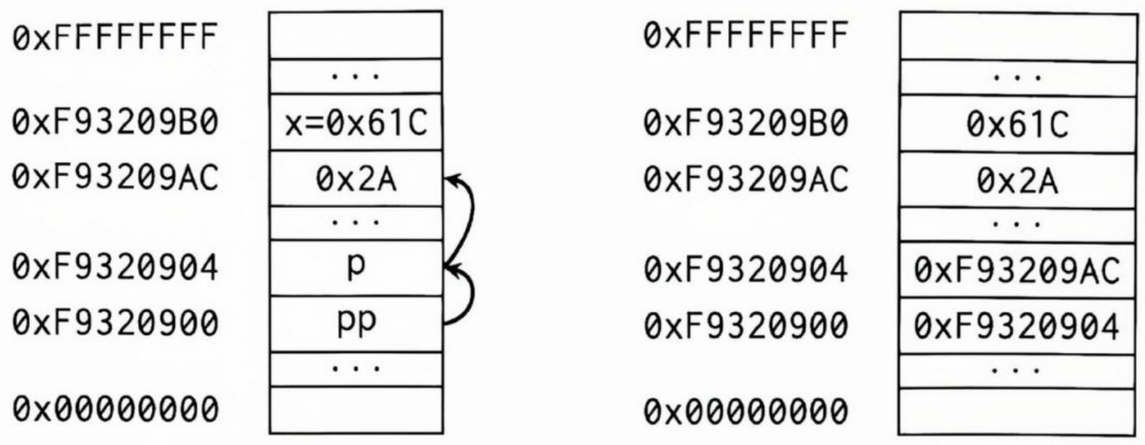
2 C 语法地

C is syntactically similar to Java, but there are a few key differences:

1. C is function-oriented, not object-oriented; there are no objects.
2. C does not automatically handle memory for you.
   - Stack memory, or *things that are not manually allocated*: data is garbage immediately after the *function in which it was defined* returns.
   - Heap memory, or *things allocated with* `malloc`, `calloc`, *or* `realloc`: data is freed only when the programmer explicitly frees it!
   - There are two other sections of memory that we learn about in this course, *static* and *code*, but we'll get to those later.
   - In any case, allocated memory always holds garbage until it is initialized!
3. C uses pointers explicitly. If `p` is a pointer, then `*p` tells us to use the value that `p` points to, rather than the value of `p`, and `&x` gives the address of `x` rather than the value of `x`.

On the left is the memory represented as a box-and-pointer diagram.

On the right, we see how the memory is really represented in the computer.



Let's assume that **int*** p is located at `0xF9320904` and **int** x is located at `0xF93209B0`. As we can observe:

- *p evaluates to `0x2A` (42_{10}).
- p evaluates to `0xF93209AC`.
- x evaluates to `0x61C`.
- &x evaluates to `0xF93209B0`.

Let's say we have an **int** **pp that is located at `0xF9320900`.

2.1 What does pp evaluate to? How about *pp? What about **pp?

pp evaluates to 0xF9320904.
*pp evaluates to 0xF93209AC.
**pp evaluates to 0x2A.

2.2 The following functions are syntactically-correct C, but written in an incomprehensible style. Describe the behavior of each function in plain English.

(a) Recall that the ternary operator evaluates the condition before the ? and returns the value before the colon (:) if true, or the value after it if false.

三元的.

```
1  int foo(int *arr, size_t n) {
2      return n ? arr[0] + foo(arr + 1, n - 1) : 0;
3  }
```

n (n!=0).

Return the sum of the first N elements in arr.

(b) Recall that the negation operator, !, returns 0 if the value is non-zero, and 1 if the value is 0. The ~ operator performs a *bitwise not* (NOT) operation.

按位取反.

```
1  int bar(int *arr, size_t n) {
2      int sum = 0, i;
3      for (i = n; i > 0; i--)
4          sum += !arr[i - 1];
5      return ~sum + 1;
6  }
```

sum：数组里面有多少个0
return "-sum"

Return -1 times the number of zeros in the first N elements of arr.

~sum + 1 → Two's Complement中取负数的方法：按位取反+1.

(c) Recall that ^ is the *bitwise exclusive-or* (XOR) operator.

按位异或（对应位不同为1，相同为0）

```
1  void baz(int x, int y) {
2      x = x ^ y;
```

2 3
x=1 3
x=1 y=1
x=0 y=1

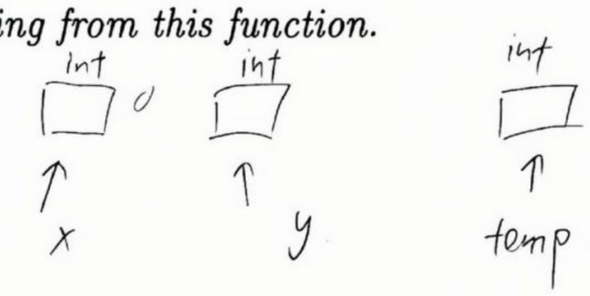
```
3       y = x ^ y;
4       x = x ^ y;      Ultimately does not change the value of either x or y.
5   }
```

(d) (Bonus: How do you write the *bitwise exclusive-nor* (XNOR) operator in C?) x==y.
对应位相同为1，不同为0.

3 Programming with Pointers

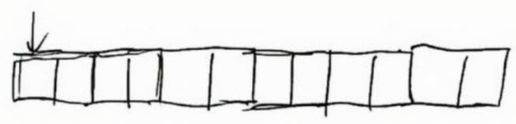
3.1 Implement the following functions so that they work as described.

(a) Swap the value of two **int**s. *Remain swapped after returning from this function.*




```
void swap( int *x , int *y )
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

(b) Return the number of bytes in a string. *Do not use* `strlen`.



```
int mystrlen( char * str ) {
    int count = 0 ;
    while ( *str++) {
        count ++;
    }
    return count ;
}
```

3.2 The following functions may contain logic or syntax errors. Find and correct them.

(a) Returns the sum of all the elements in summands.

```
1   int sum(int* summands) {
2       int sum = 0;
3       for (int i = 0; i < sizeof(summands); i++)
4           sum += *(summands + i);
5       return sum;
6   }
```

X

```
int sum(int * summands , size_t n ) {
    int sum = 0 ;
    for (int i=0 ; i<n ; i++) {
        sum += * (summands + i);
    }
    return sum;
}
```

(b) Increments all of the letters in the `string` which is stored at the front of an array of arbitrary length, `n >= strlen(string)`. Does not modify any other parts of the array's memory.

```
1   void increment(char* string, int n) {
2       for (int i = 0; i < n; i++)
```

✓ The ends of strings are denoted by the null terminator rather than n.

✓ Simply having space for n characters in the array doesn't mean the string stored inside is also of length n.

```
void increment ( char* string , int n ) {
    for (int i=0 ; string[i] != 0 ; i++)
        string [i] ++ ;  // or (*(string + i ))++ ;
}
```

```
3           *(string + i)++;
4   }
```

(c) Copies the string src to dst.

```
1   void copy(char* src, char* dst) {
2       while (*dst++ = *src++);
3   }
```

No errors.

(d) Overwrites an input string src with "61C is awesome!" if there's room. Does nothing if there is not. Assume that length correctly represents the length of src.

char * srcptr, replaceptr; initializes a char pointer, and a char. —— Not two char pointers.

```
1   void cs61c(char* src, size_t length) {
2       char *srcptr, replaceptr;
3       char replacement[16] = "61C is awesome!";
4       srcptr = src;
5       replaceptr = replacement;
6       if (length >= 16) {
7           for (int i = 0; i < 16; i++)
8               *srcptr++ = *replaceptr++;
9       }
10  }
```

* 只作用于紧跟着得变量. Correction: char * srcptr, *replaceptr.

4 Memory Management

4.1 For each part, choose one or more of the following memory segments where the data could be located: **code, static, heap, stack**.

(a) Static variables Static

(b) Local variables stack

(c) Global variables static.

(d) Constants static, stack, code.

(e) Machine Instructions code.

(f) Result of malloc heap.

(g) String Literals static, stack

字符串.字面量

4.2 Write the code necessary to allocate memory on the heap in the following scenarios

(a) An array `arr` of k integers `arr = (int*) malloc ( sizeof(int) * k );`.

(b) A string `str` containing p characters `str = (char*) malloc (sizeof(char) * (p+1));` the null terminator.

(c) An $n \times m$ matrix `mat` of integers initialized to zero. `mat = (int*) calloc( n*m, sizeof(int) );`

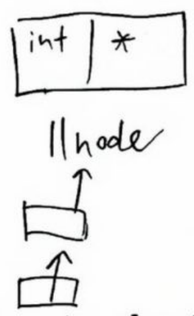
4.3 What's the main issue with the code snippet seen here? (Hint: `gets()` is a function that reads in user input and stores it in the array given in the argument.)

If the user input contains more than 63 characters, then the input will override other parts of the memory.

```
1  char* foo() {
2      char buffer[64];
3      gets(buffer);
4
5      char* important_stuff = (char*) malloc(11 * sizeof(char));
6
7      int i;
8      for (i = 0; i < 10; i++) important_stuff[i] = buffer[i];
9      important_stuff[i] = '\0';
10     return important_stuff;
11 }
```

Suppose we've defined a linked list **struct** as follows. Assume `*lst` points to the first element of the list, or is `NULL` if the list is empty.

```
struct ll_node {
    int first;
    struct ll_node* rest;
}
```



4.4 Implement `prepend`, which adds one new `value` to the front of the linked list. Hint: why use ll_node $**lst$ instead of ll_node$*lst$? → 传引用

```
void prepend(struct ll_node** lst, int value) {
    struct ll_node* new_node = (struct ll_node*) malloc ( sizeof (struct ll_node) );
    new_node -> first = value;
    new_node -> rest = *lst;
    *lst = new_node;
}
```

4.5 Implement `free_ll`, which frees all the memory consumed by the linked list.

```
void free_ll(struct ll_node** lst)
    if ( *lst ) {
        free_ll (&((*lst)->rest));
        free (*lst);
    }
    *lst = NULL;
}
```